Technische Hochschule Deggendorf
Fakultät Angewandte Informatik
Studiengang Bachelor Cybersecurity

Leichtgewichtige Kryptographie auf ressourcenbeschränkten ARM-Mikrocontrollern: Benchmark-Vergleich von Ascon-AEAD128 und AES-128-GCM

Lightweight Cryptography on Constrained ARM Microcontrollers: A Benchmark Comparison of Ascon-AEAD128 and AES-128-GCM

Projektarbeit im Kurs *Kryptologie 2*:
Bachelor of Science (B.Sc.)
an der Technischen Hochschule Deggendorf

Vorgelegt von:
Johannes Zipper
Matrikelnummer: 22411902

Prüfungsleitung:
Prof. Dr. Martin Schramm

Am: 26. Juni 2026

Abstract

This thesis presents a measurement-based comparison of Ascon-AEAD128 and AES-128-GCM on two constrained ARM Cortex-M microcontrollers representative of the automotive embedded-systems spectrum: the NUCLEO-F072RB (Cortex-M0, 48 MHz, 16 KB SRAM) and the NUCLEO-L476RG (Cortex-M4F, clocked at 48 MHz to match the F072, 128 KB SRAM). Both algorithms are evaluated as software-only implementations using the official ASCON reference implementation and mbedTLS, respectively, across message sizes from 1 to 10 000 bytes and five compiler optimization levels (`-O0` through `-Os`). Execution time is measured via a 32-bit hardware timer configured as a free-running cycle counter; code size is measured using marginal build variants with `arm-none-eabi-size`.

Ascon-AEAD128 outperforms AES-128-GCM in execution time at every measured message size, optimization level, and on both boards. At `-O2` and 1000 bytes, Ascon-AEAD128 requires 168 921 cycles on the Cortex-M0 against 770 536 for AES-128-GCM (factor 4.6), and 98 262 cycles on the Cortex-M4F against 193 311 (factor 2.0). At `-Os`, Ascon-AEAD128 consumes 4 376 bytes of Flash on the Cortex-M0 against 16 500 bytes for AES-128-GCM, a factor of approximately 3.8. The advantage of Ascon-AEAD128 is consistently larger on the more constrained Cortex-M0, and the code-size advantage holds at the size-optimized level where constrained deployments would build. These results provide a quantitative basis for preferring Ascon-AEAD128 over AES-128-GCM on constrained, software-only microcontroller targets.

Contents

List of Figures

List of Tables

1 Introduction

1.1 Cryptography in the Automotive Domain

Modern vehicles are no longer purely mechanical systems. Today's cars contain upwards of hundreds of Electronic Control Units (ECUs) responsible for functions ranging from engine management and brake control to infotainment, over-the-air software updates, and vehicle-to-everything communication. Due to the sheer number of features, these systems are increasingly interconnected. All connections can be separated into two categories: internal ones, which are mostly bus protocols such as CAN, LIN, and Automotive Ethernet, and external ones such as V2X and telematics interfaces. These circumstances increase the attack surface exposed to adversaries. A successful intrusion into a safety-relevant ECU carries consequences that extend well beyond data loss: it can compromise brake controllers, manipulate steering actuators, or falsify sensor data, directly endangering human lives.

Cryptographic protection of ECU communication is therefore not optional. Standards such as AUTOSAR SecOC (Secure Onboard Communication) and ISO/SAE 21434 establish requirements for message authentication and confidentiality across the vehicle network. In practice, the algorithm most commonly used to fulfill these requirements is the Advanced Encryption Standard (AES). It has served as the global symmetric encryption standard since its NIST standardization in 2001 [1] and is deeply embedded in automotive security stacks, ranging from hardware security modules (HSMs) to communication middleware.

1.2 Cryptographic Requirements of Modern ECUs

ECUs span a wide spectrum of computational capability. On the high end are domain controllers managing ADAS functions or running an automotive-grade operating system; these may be equipped with multi-core processors, dedicated cryptographic co-processors, and several megabytes of RAM. At the other end are simple body control modules, sensor nodes, and actuator controllers built around low-cost 32-bit microcontrollers with RAM measured in tens of kilobytes and no hardware cryptographic accelerator. On these devices the computational cost of cryptographic operations becomes a real concern.

For such constrained ECUs, the cryptographic primitive must satisfy several requirements simultaneously: it must provide authenticated encryption with associated data (AEAD) to guarantee confidentiality, integrity, and authenticity in a single pass; it must execute within a time budget that does not interfere with real-time control tasks; and its implementation must fit within the available Flash and RAM without displacing application code. These requirements motivate a closer examination of whether AES-128-GCM — designed for general-purpose hardware — remains the most appropriate choice for every node in the vehicle network, or

whether purpose-built lightweight alternatives deserve consideration.

1.3 The Limits of AES on Low-End Microcontrollers

AES-128-GCM is the dominant AEAD construction in modern security protocols and is well understood with respect to both security and implementation. On hardware that includes a dedicated AES accelerator, it executes efficiently and introduces negligible overhead. However, a large portion of automotive microcontrollers — particularly those in cost-sensitive, high-volume applications — are based on simple 32-bit cores such as the ARM Cortex-M0, which provide no hardware cryptographic acceleration. On these platforms, a software implementation of AES-128-GCM must perform the full AES key schedule and block cipher in software, and the GCM authentication step requires a 128-bit polynomial multiplication (GHASH) that maps poorly onto a minimal instruction set without hardware multiply-accumulate support. The result can be an implementation that is slower, larger in code size, or more energy-intensive than the available budget permits.

This raises a practical engineering question: for constrained ECUs where AES hardware acceleration is absent, is there a standardized alternative that offers better performance characteristics without sacrificing security?

1.4 The NIST Lightweight Cryptography Standardization Process

Recognizing the growing need for cryptographic primitives suited to constrained devices, NIST initiated a Lightweight Cryptography (LWC) standardization project in 2018 [2]. In February 2023, NIST announced ASCON as the winner [3]. The resulting standard was published in August 2025 as NIST SP 800-232 [4]. ASCON is a family of lightweight cryptographic algorithms based on a sponge construction, designed from the ground up for efficient software implementation on small processors. Its selection provides the embedded and automotive security community with a standardized, well-analyzed alternative to AES-GCM for use cases where resource constraints are the primary concern.

Despite this standardization, ASCON adoption in automotive and industrial embedded systems remains limited. AES continues to dominate deployed security stacks, in part due to inertia, existing hardware support, and a lack of quantitative performance data on representative automotive-class microcontrollers. This work aims to contribute directly to closing that gap.

1.5 Objective and Research Questions

This paper presents a quantitative benchmark comparing Ascon-AEAD128 and AES-128-GCM on two ARM Cortex-M microcontrollers representative of the low-to-mid range of the automotive ECU spectrum: the NUCLEO-F072RB (Cortex-M0, 48 MHz) and the NUCLEO-L476RG (Cortex-M4F, 80 MHz). Both boards serve as a simulation of the resource conditions found in constrained automotive ECUs. All measurements use software-only implementations to ensure a fair,

hardware-accelerator-independent comparison applicable to the broadest range of real-world deployments.

The following research questions guide the study:

1. **Performance:** How do Ascon-AEAD128 and AES-128-GCM compare in execution time and throughput across a range of message sizes on each platform?
2. **Flash footprint:** What is the Flash code-size footprint of each implementation relative to the constraints of a typical automotive microcontroller?
3. **Optimization sensitivity:** How does compiler optimization level affect the relative performance of the two algorithms?
4. **Architecture dependence:** Does the performance gap between the two algorithms differ between the Cortex-M0 and the Cortex-M4F, and what does this imply for ECU selection?

1.6 Structure of This Paper

The remainder of this paper is organized as follows. Chapter 2 provides the technical background on AES-128-GCM, the ASCON family, and the target hardware platforms. Chapter 3 describes the benchmark methodology, including the measurement setup, timing approach, library selection, and build configuration. Chapter 4 presents the measurement results. Chapter 5 discusses the findings in the context of the research questions and draws practical conclusions for algorithm selection on constrained automotive hardware. Chapter 6 concludes the paper and outlines directions for future work.

2 Background

2.1 Authenticated Encryption with Associated Data (AEAD)

Cryptographic protocols for embedded communication commonly require three security objectives simultaneously: confidentiality (to prevent unauthorized reading of the message), integrity (to detect any modification during transmission), and authenticity (to verify that the message originates from a legitimate sender). Traditionally, these properties were achieved by combining a separate cipher with a message authentication code (MAC). However, the secure composition of two independent primitives is non-trivial: the order of operations matters, and incorrect combinations can introduce vulnerabilities even when each primitive is individually sound. Authenticated Encryption with Associated Data (AEAD) addresses this by combining all three objectives into a single, unified primitive.

An AEAD scheme takes four inputs: a secret key K, a nonce N (a value that must be unique per encryption under a given key), a plaintext message M, and a block of associated data AD. The associated data is authenticated but not encrypted: it remains readable in the clear while being bound to the ciphertext via the authentication tag. The scheme produces a ciphertext C and an authentication tag T. Decryption takes the same key and nonce, as well as the ciphertext, tag, and associated data, and returns the recovered plaintext only if the tag verifies successfully; otherwise it returns an explicit failure. This fail-closed behavior is essential in embedded contexts, since a receiver that processes unauthenticated data before verification completes is vulnerable to plaintext-dependent side channels and fault injection.

For the constrained microcontroller setting considered in this work, two AEAD properties are of particular practical relevance. First, single-pass processing absorbs plaintext and produces the authentication tag in one sequential pass over the data, minimizing RAM requirements by avoiding the need to buffer the full message. Second, a compact state and key schedule reduce code size and initialization overhead, both of which are essential on devices with limited Flash. These properties will later serve as evaluation criteria when comparing AES-128-GCM and Ascon-AEAD128.

2.2 AES-128-GCM

2.2.1 The AES Block Cipher

The Advanced Encryption Standard (AES) is a substitution-permutation block cipher standardized by NIST in 2001 [1]. It operates on a fixed block size of 128 bits and supports key lengths of 128, 192, or 256 bits; this work uses the 128-bit key variant exclusively (AES-128). The cipher applies ten rounds of transformation to a 4×4 byte state matrix. Each round consists of four operations: `SubBytes` (a non-linear byte substitution via the AES S-box), `ShiftRows` (a cyclic

shift of state rows), `MixColumns` (a linear diffusion step operating on state columns in $\mathrm{GF}(2^8)$), and `AddRoundKey` (XOR of the state with a round-specific subkey). The 128-bit key is expanded into eleven round keys before encryption begins, a process known as the key schedule.

The security of AES rests on more than two decades of public cryptanalysis; no practical attack against the full cipher is known. With hardware acceleration (a dedicated AES engine), the cipher executes at throughputs measured in gigabits per second. On a software-only Cortex-M0 implementation, however, the `MixColumns` step and the S-box lookups impose a considerable cycle cost, because the M0 lacks the wider data path and lookup acceleration found on higher-end cores. This performance gap on minimal instruction-set architectures is the central motivation for evaluating a lightweight alternative.

2.2.2 Galois/Counter Mode (GCM)

AES alone is a block cipher and not an AEAD scheme: it encrypts exactly one 128-bit block and provides no authentication. Galois/Counter Mode (GCM) combines AES in counter mode (CTR) for encryption with a polynomial hash (GHASH) for authentication, yielding an AEAD construction [5].

In CTR mode, a counter value is encrypted by AES and the resulting keystream block is XORed with the plaintext. The counter is incremented for each successive block, which enables parallelized encryption independent of plaintext content. GHASH computes a 128-bit authentication tag using polynomial operations over the ciphertext blocks and associated data in $\mathrm{GF}(2^{128})$, a finite field defined over 128-bit values. The hash key H is derived by encrypting a zero block under the session key at initialization.

GCM's parallelizability is its primary advantage on general-purpose hardware with wide data paths. Its principal weakness on constrained processors is the GHASH step: multiplication in $\mathrm{GF}(2^{128})$ requires either a hardware carry-less multiply instruction or a software implementation using multiple 32-bit operations. The Cortex-M0 provides only a 32-bit multiply without a carry-less variant, so a software GHASH implementation on that core requires a series of XOR and shift operations that contribute noticeably to the total cycle count. Additionally, GCM requires a unique nonce per encryption; nonce reuse under the same key is catastrophic, recovering both the hash key and the plaintext with a small number of observed ciphertexts.

2.2.3 Software-Only Scope

The STM32L476RG does not include a hardware AES peripheral; that block is present only in the STM32L486 and related parts of the same product family, as confirmed by the device reference manual. Both benchmark boards therefore present an identical software-only execution environment for this comparison. The absence of a hardware cryptographic accelerator on the selected boards is a deliberate design choice: restricting the comparison to software implementations throughout makes the results applicable to the broad class of constrained 32-bit microcontrollers that ship without vendor-specific AES acceleration, which represents the majority of cost-sensitive automotive nodes.

2.3 The ASCON Family

2.3.1 The ASCON Permutation

All ASCON variants use the same underlying permutation, written as p^a or p^b depending on how many rounds are applied. The permutation operates on a 320-bit state organized as five 64-bit words x_0 through x_4. Each round applies three layers in sequence.

The first layer, p_C (constant addition), XORs a round-specific constant into word x_2, introducing asymmetry between rounds to prevent slide attacks. The second layer, p_S (substitution), applies a 5-bit S-box in parallel to all 64 bit-sliced positions across the five state words. This S-box provides non-linearity with simple arithmetic and was chosen for the efficiency of its bitsliced implementation on processors with native 64-bit or 32-bit word operations. The third layer, p_L (linear diffusion), applies independent linear transformations to each of the five state words using rotations and XOR, ensuring that differences propagate rapidly across the full state.

The full ASCON permutation uses twelve rounds (p^{12}) for initialization and finalization. The internal data-processing step uses eight rounds (p^8) for Ascon-AEAD128. On 32-bit microcontrollers the five 64-bit state words are held in pairs of 32-bit registers, so the rotation-and-XOR structure of the linear layer maps efficiently onto the available register file without memory traffic.

2.3.2 Sponge and Duplex Construction

ASCON uses a sponge-based construction — specifically the duplex variant — rather than the block-cipher-mode approach used by AES-GCM. The sponge model divides the state into two parts: the rate r (the part that interfaces with input and output) and the capacity c (the part that remains hidden from I/O, forming the security margin). For Ascon-AEAD128, the rate is 128 bits and the capacity is 192 bits, giving a total state of 320 bits.

In the duplex variant used for AEAD, plaintext is XORed into the rate portion of the state one rate-sized block at a time, followed by the permutation. The ciphertext is extracted from the state after the absorption, mixing encryption and authentication in a single sequential pass. This single-pass property means that memory buffering of the full message is never required, a significant advantage over constructions that must process the ciphertext twice (once for decryption, once for authentication).

The capacity portion of the state is never directly observable by an attacker and provides the security guarantee. For Ascon-AEAD128, the 192-bit capacity yields a claimed security level of 128 bits against generic attacks on both confidentiality and integrity.

2.3.3 Ascon-AEAD128

Ascon-AEAD128 is the primary AEAD member of the ASCON family and the algorithm selected as the winner of the NIST Lightweight Cryptography standardization process [3]. It is specified as a standard in NIST SP 800-232 [4]. It accepts a 128-bit key, a 128-bit nonce, associated data of arbitrary length, and a plaintext of arbitrary length, and produces a ciphertext of the same length as the plaintext together with a 128-bit authentication tag.

The processing pipeline proceeds in four phases. In the initialization phase, the 320-bit state is assembled from the key, nonce, and algorithm-specific initialization vector, then transformed by p^{12}. In the associated data phase, the associated data is absorbed into the rate in 128-bit blocks, each followed by p^8; the last block is padded, and a domain separation constant is XORed into the state at the end of this phase. In the encryption phase, plaintext blocks are absorbed and ciphertext blocks are extracted in the same pass, each followed by p^8. In the finalization phase, the key is XORed into the state, p^{12} is applied, and the authentication tag is extracted from the last 128 bits of the state.

For the benchmark in this work, Ascon-AEAD128 is evaluated using the official ASCON reference implementation in C [6].

2.3.4 Ascon-Hash256

Ascon-Hash256 is the hash function member of the ASCON family. It produces a 256-bit digest from an input of variable length using the same p^{12} permutation as Ascon-AEAD128. Message absorption proceeds in 64-bit rate blocks, each followed by p^{12}, and the digest is squeezed in two 64-bit output blocks after all input has been absorbed.

2.3.5 Ascon-80pq and the Post-Quantum Outlook

Ascon-80pq is a variant of Ascon-AEAD128 that extends the key length from 128 to 160 bits while keeping the nonce at 128 bits and the tag at 128 bits. The longer key is a precautionary measure against Grover's algorithm, which effectively halves the security level of a symmetric key in bits against an adversary with a large-scale quantum computer. A 128-bit key offers approximately 64-bit post-quantum security under Grover's model, while the 160-bit key of Ascon-80pq provides approximately 80-bit post-quantum security.

The performance overhead of the 160-bit key variant relative to Ascon-AEAD128 is limited to initialization, where the additional 32 bits are incorporated into the state setup. The permutation rounds and data processing phases are identical. Ascon-80pq is therefore of interest as a forward-compatible migration path for automotive security architectures that must account for long vehicle service lifetimes, during which the practical capabilities of quantum computing may evolve. Ascon-80pq is part of the original ASCON v1.2 competition submission [7] and is not included in NIST SP 800-232, which standardizes only Ascon-AEAD128, Ascon-Hash256, Ascon-XOF128, and Ascon-CXOF128 [4].

2.4 Architectural Comparison: Sponge vs. Block Cipher + GCM

The structural differences between AES-128-GCM and Ascon-AEAD128 have concrete implications for software performance on constrained microcontrollers, which are worth making explicit before the benchmark results are presented.

AES-128-GCM is built from two largely independent components: the AES block cipher and the GHASH authenticator. Each has its own state, key material, and processing requirements. The AES key schedule expands 128 bits into 176 bytes of round keys that must either be precomputed and stored in RAM or recomputed for each execution. The GHASH state requires

an additional 128-bit accumulator and the hash key H. On a Cortex-M0 without hardware multiply, GHASH is the performance bottleneck for short messages, since its initialization cost (one AES block encryption) is spread poorly at small input sizes.

Ascon-AEAD128 holds its entire state in the 320-bit permutation input. There is no separate key schedule: the key is absorbed directly into the state during initialization. The permutation is defined entirely by bitwise rotations and XOR, operations that are cheap on any 32-bit or 64-bit processor without specialized hardware. The absence of S-box table lookups also eliminates a class of cache-timing side channels that must be considered when deploying AES on processors without constant-time hardware support.

Both algorithms process data in 128-bit units per core operation: AES encrypts one 128-bit block per cipher call, and Ascon-AEAD128 absorbs one 128-bit rate block per permutation call. The throughput difference between them therefore cannot be attributed to a rate disadvantage on either side and must be found in the per-operation cost of the underlying primitive. Whether Ascon-AEAD128 or AES-128-GCM requires fewer cycles on a given platform cannot be determined theoretically; answering it quantitatively is the central goal of this benchmark.

2.5 Target Platforms

2.5.1 ARM Cortex-M0 — NUCLEO-F072RB

The NUCLEO-F072RB board is built around the STM32F072RB microcontroller, which integrates an ARM Cortex-M0 core clocked at 48 MHz. The Cortex-M0 is the smallest and most constrained member of the ARM Cortex-M family. It implements the ARMv6-M instruction set architecture, a minimal 16/32-bit Thumb instruction set without the DSP extensions, floating-point unit, or hardware divide instruction present in larger M-class cores. The core has no instruction or data caches.

Relevant characteristics for this benchmark include the absence of a hardware multiply-accumulate that would accelerate GHASH, a single-cycle 32×32-bit multiply (which produces a 32-bit result), and the lack of a Data Watchpoint and Trace (DWT) unit with a cycle counter. Because the DWT cycle counter is not present on the M0, cycle-accurate measurement on this board uses TIM2, configured as a 32-bit free-running counter driven by the system clock.

The F072RB is representative of the lowest tier of 32-bit microcontrollers found in automotive body and sensor applications, where unit cost and power consumption are the dominant design constraints.

2.5.2 ARM Cortex-M4F — NUCLEO-L476RG

The NUCLEO-L476RG board is built around the STM32L476RG microcontroller, which integrates an ARM Cortex-M4F core clocked at up to 80 MHz. The Cortex-M4F implements the ARMv7E-M instruction set architecture, which extends ARMv6-M with DSP-oriented instructions including a single-cycle 32×32-bit multiply producing a 64-bit result (`UMULL`, the variant relevant to GHASH) and 64-bit multiply-accumulate (`UMLAL`, `SMLAL`), single-instruction saturating arithmetic, and a single-precision floating-point unit (FPU). The core also includes an Adaptive Real-Time (ART)

memory accelerator, which implements an instruction prefetch buffer and a branch cache to reduce Flash access latency at high operating frequencies.

For timing, the Cortex-M4F provides a DWT unit including the `DWT->CYCCNT` cycle counter, a 32-bit register incremented on every processor clock, readable with a single memory-mapped register access.

The L476RG is representative of a mid-range automotive microcontroller, suitable for body control modules and sensor fusion nodes with moderate processing requirements. Together with the F072RB, the two boards span a performance range relevant to the lower half of the automotive ECU spectrum.

2.6 Related Work

Performance comparisons between ASCON and AES-GCM on embedded hardware have appeared in several contexts since ASCON's selection as the NIST LWC standard. Benchmarks conducted on AVR 8-bit and MSP430 16-bit platforms during the NIST LWC competition evaluation process demonstrated ASCON's advantage on ultra-constrained targets, but these results are not directly applicable to 32-bit ARM Cortex-M deployments that represent the current generation of automotive microcontrollers.

Work targeting ARM Cortex-M platforms specifically has been published both by the ASCON team and by independent researchers. The ASCON team provides cycle counts for their reference and optimized implementations on several Cortex-M targets [6]. However, published figures often report single-point measurements at a fixed message length rather than throughput curves across a logarithmic range of input sizes, which are needed to characterize behavior for the short messages typical in automotive CAN frames and sensor update packets.

To the authors' knowledge, no published work directly compares AES-128-GCM and Ascon-AEAD128 across both a Cortex-M0 and a Cortex-M4F target with a controlled, software-only methodology that includes compiler optimization level as an explicit variable. This work aims to fill that gap by providing reproducible benchmark data for both platforms under a unified measurement framework.

3 Methodology

3.1 Overview and Experimental Design

This chapter describes how the benchmark was constructed and how every reported number was produced. The goal is a fair and reproducible comparison of Ascon-AEAD128 and AES-128-GCM under matched conditions on two constrained microcontrollers. Fairness here has a precise meaning: both algorithms run as software-only implementations, both are measured through an identical timing harness, both are compiled with the same toolchain and the same set of optimization levels, and the one-time setup cost is excluded from the timed region for both in the same way.

The experimental design varies four factors: the algorithm (Ascon-AEAD128 or AES-128-GCM), the hardware platform (Cortex-M0 or Cortex-M4F), the input message size (swept across several orders of magnitude), and the compiler optimization level (swept from no optimization through size optimization). Each combination is measured many times and reduced to summary statistics offline. Code size is measured separately, as a static property of the compiled binaries rather than a runtime measurement.

3.2 Target Platforms

Two STMicroelectronics Nucleo-64 development boards represent the low and middle of the constrained range. Their relevant properties are summarized in Table 3.1. Both boards expose an on-board ST-LINK debugger with a USB virtual COM port routed to USART2, which carries the measurement output.

Table 3.1: Target platform properties

Property	NUCLEO-F072RB	NUCLEO-L476RG
Microcontroller	STM32F072RBT6	STM32L476RGT6
Core	Cortex-M0	Cortex-M4F
Maximum clock	48 MHz	80 MHz
Flash	128 KB	1 MB
SRAM	16 KB	128 KB
Hardware AES	None	None
Cycle counter (DWT)	Absent	Present
Instruction prefetch	None	ART accelerator
Floating-point unit	None	Single precision

3.2.1 NUCLEO-F072RB (Cortex-M0)

The F072 carries a Cortex-M0 core with no hardware multiplier for wide operands, no instruction cache, no branch predictor, and no floating-point unit. It represents the lower end of the device range, where every cycle and every kilobyte is scarce. Its 16 KB of SRAM is the binding constraint on this study, and it limits the largest message size that can be benchmarked on this board.

3.2.2 NUCLEO-L476RG (Cortex-M4F)

The L476 carries a Cortex-M4F core with a single-cycle hardware multiplier, a single-precision floating-point unit, and the ST ART accelerator, which prefetches and caches instructions from Flash. It represents the middle of the range. The L476 has substantially more memory than the F072, so memory is not a constraint on this board, but it is included to expose how the relative performance of the two algorithms depends on the processor architecture.

The specific part on the NUCLEO-L476RG does not contain a hardware AES block; only the L486 and related parts in the same family include one. This absence is useful for the study, because it means there is no hardware cryptographic path that could be engaged accidentally, so the software comparison is consistent on both boards.

3.2.3 Clocking and the Matched-Frequency Decision

Both boards run at 48 MHz for the timing comparison. The L476 is capable of 80 MHz, so it is intentionally underclocked to the F072 ceiling. This choice removes clock frequency as a variable: any difference in measured time between the two boards at the same clock then reflects the processor architecture alone, not a difference in operating frequency.

The measurements are reported primarily as cycle counts rather than wall-clock time, since the timer increments once per processor clock (see Section 3.6.1), so one timer tick equals one clock cycle and the cycle count is independent of the operating frequency.

3.3 Toolchain and Build System

All firmware was built with a single toolchain to keep the comparison consistent. The compiler is `arm-none-eabi-gcc` version 14.3.1, supplied with the STM32CubeCLT bundle. The build is driven by CMake with the Ninja generator, both bundled with CubeCLT. Project skeletons, including peripheral initialization and the linker script, were generated with STM32CubeMX configured for CMake output. The host operating system was Windows.

The base compiler flags follow the CubeMX template. They include `-ffunction-sections`, `-fdata-sections`, and linker garbage collection (`-Wl,-gc-sections`), the size-oriented newlib-nano C library (`-specs=nano.specs`), and the architecture flags appropriate to each core. On the L476 these include the hardware floating-point ABI, although floating-point arithmetic is not used by either algorithm.

The optimization level is the one build variable that changes across the measurement matrix. It is controlled through a single CMake cache variable set per build preset. Each preset reconfigures the build at one optimization level, from `-O0` through `-Os`, so that the same source produces a family of binaries that differ only in optimization.

3.4 Cryptographic Implementations

Both implementations are taken from existing open-source projects rather than written for this study. This keeps the comparison representative of what a developer would actually deploy, and it avoids the risk of an unrepresentative hand-written implementation favoring one side.

3.4.1 AES-128-GCM (mbedTLS)

AES-128-GCM is provided by mbedTLS version 3.6.x [8]. mbedTLS was selected because it is a widely deployed, well-maintained library with a complete and verified AES-GCM implementation, which lends the comparison credibility. The library was reduced to a minimal configuration containing only the modules required for AES-128-GCM.

Three configuration choices affect the measurements and are disclosed here, because the reported numbers are only interpretable alongside them.

First, the small four-bit GHASH multiplication table is used. The larger eight-bit table is faster per byte but enlarges the GCM context by roughly 3.8 KB of RAM. The small table was chosen because it represents a genuinely lightweight configuration of AES-GCM, matching the constrained-device framing of this work. As a consequence, the reported AES-GCM throughput represents the memory-frugal end of what mbedTLS can achieve rather than its speed ceiling. This point is revisited in the discussion.

Second, the AES lookup tables are placed in Flash rather than generated in RAM at initialization. This shifts roughly 8 KB from RAM to Flash, which is the realistic trade-off on a part with only 16 KB of SRAM. It affects the code-size results but has no meaningful effect on runtime.

Third, only the 128-bit key length is compiled. The 192-bit and 256-bit key schedules are removed. Because only AES-128 is benchmarked, this has no effect on runtime and only a small effect on Flash size.

3.4.2 Ascon-AEAD128 (Reference Implementation)

Ascon-AEAD128 is provided by the official ASCON reference implementation in C [6]. The reference variant was chosen over the optimized variants for portability and because it corresponds directly to the standardized specification. The implementation uses a 16-byte key, a 16-byte nonce, and produces a 16-byte authentication tag, with a rate of 16 bytes.

The choice of the reference variant has a visible effect on code size at high optimization levels. This is reported and discussed rather than hidden, because it reflects what a developer integrating the standard reference code would observe.

3.4.3 Uniform Wrapper Interface

Each implementation is reached through a thin wrapper with a matching signature, so that the benchmark harness calls both algorithms the same way. The AES wrapper separates key setup from encryption, so that the one-time key schedule can be hoisted out of the timed region. The ASCON single-pass AEAD interface has no separable key schedule, so its encryption call is timed in full. Neither wrapper performs additional copying inside the timed path beyond what the underlying library requires.

3.5 Correctness Verification

No timing measurement was trusted until the implementation producing it was shown to be correct. Each algorithm was verified against published test vectors before any benchmark was run.

AES-128-GCM was checked against a NIST Known Answer Test vector, confirming both the ciphertext and the authentication tag for a known key, nonce, and input. It was additionally checked with a round-trip test, where decryption of the produced ciphertext returns the original plaintext, and with a forgery-rejection test, where a modified tag is correctly rejected.

Ascon-AEAD128 was checked against a vector from the Lightweight Cryptography Known Answer Test set, confirming the combined ciphertext and tag for a known key and nonce. It was likewise checked with a round-trip test and a forgery-rejection test.

Both verification routines run on the target hardware at startup, before the benchmark begins, and print a pass result over the serial link. A failure halts interpretation of any subsequent timing for that build.

3.6 Measurement Methodology

3.6.1 Timing Source

Both boards use the TIM2 timer as the timing source. TIM2 is a 32-bit timer on both families, configured identically on each board as a free-running up-counter driven by the internal clock with the prescaler set to zero. With a zero prescaler the counter increments once per processor clock cycle, so one tick equals one cycle. At 48 MHz the 32-bit counter wraps approximately every 89 seconds, which is far longer than any single measurement. All elapsed-time computations use unsigned 32-bit subtraction, which handles a single wrap during a measurement correctly without special-case logic.

The Cortex-M0 on the F072 does not provide the Data Watchpoint and Trace cycle counter available on the Cortex-M4F. Using TIM2 on both boards therefore gives a comparison between identically configured peripherals rather than between two architecturally different timing mechanisms.

3.6.2 Interrupt Masking and Warmup

The benchmark macro disables all maskable interrupts for the duration of the measured region and re-enables them afterward. This prevents the periodic system tick interrupt and any peripheral interrupts from polluting short measurements. The effect was confirmed during validation, where masking reduced the per-iteration variation from roughly 28 ticks to zero on a reference workload.

Each measurement runs the workload once before the timed iterations begin, and the result of this first run is discarded. This warmup primes the L476 ART accelerator and any related microarchitectural state. On the L476 the first run differs from the steady state by roughly 1.5% at small sizes. On the F072 there is no measurable warmup effect, because the Cortex-M0 has no instruction cache or branch predictor that could be cold.

3.6.3 Defeating Dead-Code Elimination

At higher optimization levels the compiler can remove a computation whose result is never used. To prevent this, each workload writes one byte of its output into a `volatile` variable. The `volatile` qualifier forces the compiler to produce the output, because it cannot prove the write is unnecessary. This protection is verified indirectly by checking that the smallest input size still records a non-zero time at every optimization level.

3.6.4 Scope of the Timed Region

The timed region contains only the authenticated-encryption operation. For AES-128-GCM the key schedule is performed once before the measurements begin and is not included. For Ascon-AEAD128 the single-pass interface performs key setup as part of the encryption call and cannot separate it, so the full call is timed. This treats the one-time setup symmetrically: the repeated per-call cost is measured for both, and the fixed setup that a real deployment would perform once is excluded from both where the interface allows.

All measurements encrypt plaintext with no associated data.

3.6.5 Output Format and Offline Statistics

Each measured iteration is streamed over the serial link as a row of comma-separated values containing the algorithm, the board, the clock frequency, the input size, the iteration index, and the measured ticks. Summary statistics — namely the median, minimum, maximum, and interquartile range — are computed offline rather than on the device. This keeps the on-device harness minimal and allows the analysis to be revised without re-running the measurements.

Under interrupt masking the per-iteration variation was zero on both boards for all measured workloads, so the median, minimum, and maximum coincide.

3.7 Benchmark Parameters

3.7.1 Input Sizes

Message sizes are swept on a logarithmic scale. On the L476 the sizes are 1, 10, 100, 1000, and 10 000 bytes. On the F072 the largest size is omitted, because the buffers required for a 10 000-byte message do not fit within 16 KB of SRAM, so the F072 sizes are 1, 10, 100, and 1000 bytes. The logarithmic spread separates the fixed per-call cost, which dominates at small sizes, from the per-byte cost, which dominates at large sizes.

Each size is measured for 100 iterations after the discarded warmup run.

3.7.2 Optimization-Level Matrix

Each algorithm on each board is compiled at five optimization levels: `-O0`, `-O1`, `-O2`, `-O3`, and `-Os`. This sweep shows how much of the measured performance depends on optimization rather than on the algorithm itself, and it identifies the level at which each algorithm performs best,

which differs by algorithm and by architecture. Reporting only a single optimization level would hide both effects.

3.8 Footprint Measurement

Code size is measured as a static property of the compiled binaries using `arm-none-eabi-size`, which reports the size of the program text, the initialized data, and the zero-initialized data sections. Flash usage is taken as the sum of the text and initialized-data sections.

Because a single binary links both algorithms together with the common platform and harness code, the size of one combined binary cannot attribute code to one algorithm or the other. To obtain a per-algorithm figure, three build variants are produced from the same source through a compile-time selection: a baseline variant (platform and harness code, no cryptographic implementation), an AES variant (adds AES-128-GCM), and an ASCON variant (adds Ascon-AEAD128). The marginal cost of each algorithm is the difference between its variant and the baseline.

Footprint is reported primarily at the size-optimization level, because that is the configuration a memory-constrained deployment would use. Footprint at the other optimization levels is also recorded, as the code size of the ASCON reference implementation varies considerably with optimization.

3.9 Harness Validation

Before any cryptographic code was measured, the harness was validated with a `memcpy` workload of known behavior, measured at sizes 1, 10, 100, and 1000 bytes for 100 iterations each. Four checks were defined: the measured time should grow roughly in proportion to the input size; the per-iteration variation should be at most a few ticks; the smallest input size should record a non-zero time; and the reported clock frequency should read 48 MHz on both boards.

All four checks passed on both boards. The per-iteration variation was zero on both boards after the warmup run. The measured ratio between the two boards for this memory workload settled at approximately 2.8 to 1 at the largest size, which reflects the architectural cost of the Cortex-M0 relative to the Cortex-M4F at the same clock. This figure serves as a reference point for interpreting the cryptographic ratios between the two boards.

3.10 Reproducibility and Known Limitations

The exact mbedTLS configuration is committed to the repository, and the commit hashes of mbedTLS and the ASCON reference implementation are recorded [8, 6]. The compiler, CMake, Ninja, and CubeCLT versions are recorded, as are the board hardware revisions and the ST-LINK firmware versions.

Several limitations are disclosed for honesty of interpretation. The reported times include a small fixed overhead from the timer read pair, the loop branch, and the `volatile` store. At the largest size this overhead is on the order of one percent of the total; at the smallest size it dominates, so the smallest-size figures should be read as a measurement of fixed per-call

cost rather than of encryption work. The harness does not subtract this overhead, and raw ticks are reported as measured. The AES-GCM measurements use the small GHASH table, as disclosed in Section 3.4.1, and therefore represent the memory-frugal end of AES-GCM rather than its fastest configuration. The ART accelerator is left enabled on the L476, which is the realistic deployment configuration, and cycle counts are reported so that the comparison does not depend on its effect.

4 Results

4.1 Overview

This chapter presents the measured results in three parts. Section 4.2 reports the harness validation, which establishes that the timing infrastructure is trustworthy. Section 4.3 reports the execution-time results across message sizes and optimization levels. Section 4.4 reports the throughput results in cycles per byte. Section 4.5 reports the code-size results. All timing figures are cycle counts at 48 MHz, where one timer tick equals one processor cycle. All results derive from the single consolidated data set, and every figure and table is reproducible from it.

4.2 Harness Validation

The `memcpy` validation passed all four checks on both boards. The measured time grew in proportion to the input size, the per-iteration variation was zero after the discarded warmup run, the smallest input size recorded a non-zero time, and both boards reported the expected 48 MHz clock. The per-iteration determinism observed here persisted through all later measurements: across the cryptographic benchmarks the median, minimum, and maximum coincided for every measured point, so the timing results are reported as single median values without dispersion.

The ratio between the two boards for the `memcpy` workload settled at approximately 2.8 to 1 at the largest size, with the Cortex-M0 slower than the Cortex-M4F at the same clock. This figure is the architectural baseline against which the cryptographic ratios in the following sections are read.

4.3 Execution Time

4.3.1 Runtime Across Message Sizes

Figure 4.1 shows median execution time against input size on logarithmic axes, with one panel per board, at optimization level `-O2`. On both boards Ascon-AEAD128 is faster than AES-128-GCM at every measured size. The curves share a characteristic shape: at the smallest sizes the time is nearly constant, because the fixed per-call cost dominates; above approximately 100 bytes the time rises in proportion to the input size, which is the per-byte encryption cost. The vertical separation between the two algorithms is present across the whole range and is wider on the Cortex-M0 board than on the Cortex-M4F board.

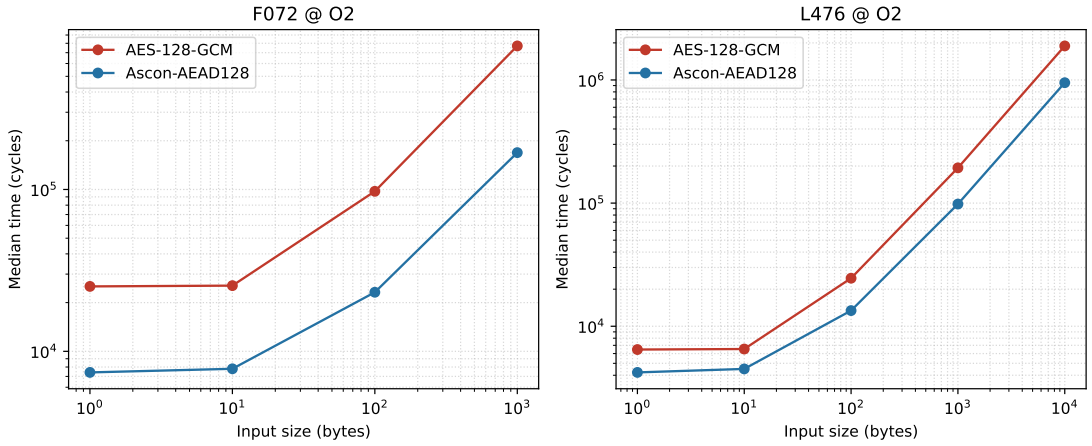

Figure 4.1: Median execution time vs. message size at `-O2` on both boards (logarithmic axes). Ascon-AEAD128 is consistently faster than AES-128-GCM at every size.

4.3.2 Effect of Optimization Level

Figure 4.2 shows median execution time at a fixed message size of 1000 bytes across the five optimization levels, with one panel per board and a logarithmic time axis. Table 4.1 gives the underlying values.

The dominant effect is the step from `-O0` to `-O1`. On the Cortex-M0 the Ascon-AEAD128 time at 1000 bytes falls from 1,387,063 cycles at `-O0` to 170,741 cycles at `-O1`, a factor of about eight. The AES-128-GCM time falls from 2,661,457 to 792,911 cycles over the same step, a factor of about three. On the Cortex-M4F the same step takes Ascon-AEAD128 from 726,789 to 104,422 cycles and AES-128-GCM from 845,537 to 242,980 cycles.

Between `-O1`, `-O2`, and `-O3` the changes are smaller and not uniform. On the Cortex-M4F, AES-128-GCM is essentially unchanged from `-O2` to `-O3` (193,311 and 193,283 cycles), while Ascon-AEAD128 improves from 98,262 to 70,194 cycles. On the Cortex-M0, AES-128-GCM improves notably at `-O3`, from 770,536 to 485,227 cycles, while Ascon-AEAD128 improves modestly, from 168,921 to 145,888 cycles.

At `-Os` both algorithms are slower than at `-O2` on both boards. On the Cortex-M0 the Ascon-AEAD128 time at `-Os` is 268,899 cycles, slower than its `-O1` time. On the Cortex-M4F the corresponding `-Os` time is 139,993 cycles.

At `-O0` the two algorithms are close, especially on the Cortex-M4F where the ratio is 1.16. Once optimization is enabled the ratio is larger, and at every optimized level the ratio is greater on the Cortex-M0 than on the Cortex-M4F. The largest ratio in favor of Ascon-AEAD128 occurs at `-O1` and `-O2` on the Cortex-M0, at approximately 4.6.

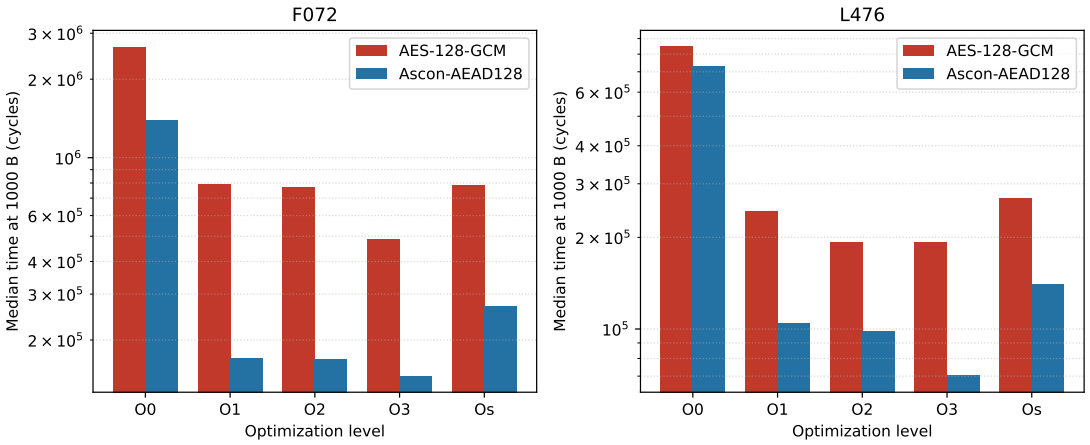

Figure 4.2: Median execution time at 1000 bytes across optimization levels on both boards (logarithmic time axis).

Table 4.1: Median execution time (cycles) and ratio at 1000 bytes per optimization level.

Level	M0 ASCON	M0 AES	M0 ratio	M4 ASCON	M4 AES	M4 ratio
`-O0`	1,387,063	2,661,457	1.92	726,789	845,537	1.16
`-O1`	170,741	792,911	4.64	104,422	242,980	2.33
`-O2`	168,921	770,536	4.56	98,262	193,311	1.97
`-O3`	145,888	485,227	3.33	70,194	193,283	2.75
`-Os`	268,899	787,284	2.93	139,993	269,250	1.92

4.4 Throughput

Figure 4.3 shows median cycles per byte against input size on logarithmic axes, with one panel per board, at `-O2`. At the smallest size the cost per byte is highest, because the fixed per-call cost is divided across a single byte. As the message grows the cost per byte falls toward a flat asymptote, which is the per-byte encryption rate.

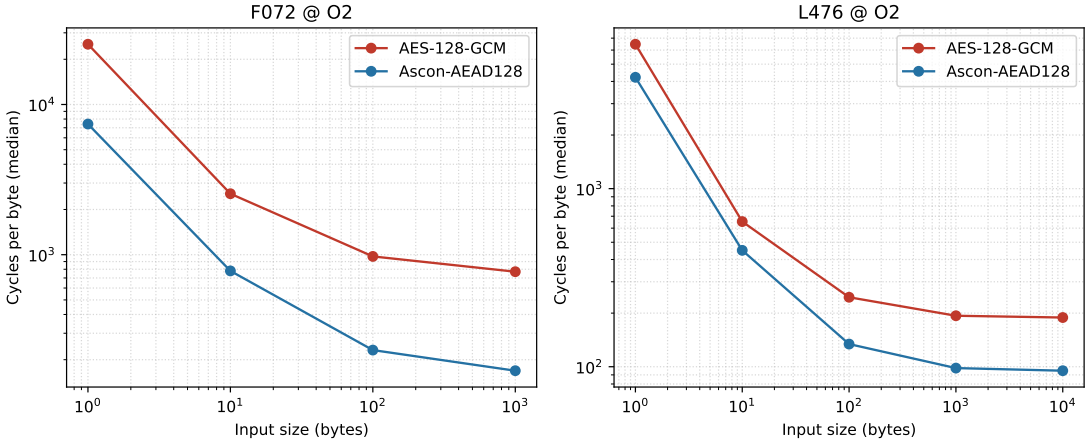

Figure 4.3: Median cycles per byte vs. message size at `-O2` on both boards (logarithmic axes). The asymptotic value at large sizes reflects the steady-state per-byte encryption rate.

In the large-message region the per-byte rates at `-O2` are as follows. On the Cortex-M0 at 1000 bytes, Ascon-AEAD128 reaches approximately 169 cycles per byte and AES-128-GCM approximately 771 cycles per byte. On the Cortex-M4F at 10,000 bytes, Ascon-AEAD128 reaches approximately 95 cycles per byte and AES-128-GCM approximately 189 cycles per byte.

At the smallest message size of one byte, where the figure reflects the fixed per-call cost, the values at `-O2` are 7,410 cycles for Ascon-AEAD128 and 25,185 cycles for AES-128-GCM on the Cortex-M0, and 4,219 cycles for Ascon-AEAD128 and 6,470 cycles for AES-128-GCM on the Cortex-M4F.

4.5 Code Size

Figure 4.4 shows the marginal Flash cost of each algorithm at `-Os` for both boards. Table 4.2 gives the marginal Flash cost at every optimization level.

At `-Os`, Ascon-AEAD128 adds 4,376 bytes of Flash on the Cortex-M0 and 4,208 bytes on the Cortex-M4F. AES-128-GCM adds 16,500 bytes on the Cortex-M0 and 15,776 bytes on the Cortex-M4F. The ratio of AES-128-GCM to Ascon-AEAD128 marginal Flash is approximately 3.8 on both boards.

The Ascon-AEAD128 marginal Flash cost varies strongly with optimization level. Figure 4.5 shows this for each board. At `-Os` it is the smaller of the two algorithms by a wide margin. At

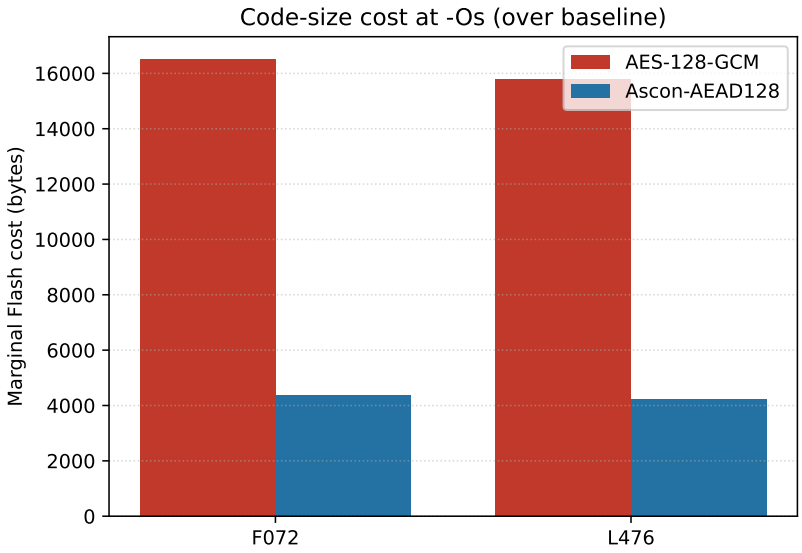

Figure 4.4: Marginal Flash cost at `-Os` for each algorithm and board. Ascon-AEAD128 requires approximately 3.8× less Flash than AES-128-GCM at this level.

Table 4.2: Marginal Flash cost (bytes) and ratio per optimization level.

Level	M0 AES	M0 ASCON	M0 ratio	M4 AES	M4 ASCON	M4 ratio
`-O0`	26,296	12,836	2.05	23,256	6,776	3.43
`-O1`	17,396	27,924	0.62	16,496	24,208	0.68
`-O2`	17,476	27,504	0.64	16,584	23,264	0.71
`-O3`	24,208	43,024	0.56	17,720	34,560	0.51
`-Os`	16,500	4,376	3.77	15,776	4,208	3.75

`-O1`, `-O2`, and `-O3` it is larger than the AES-128-GCM cost, reaching a maximum at `-O3` of 43,024 bytes on the Cortex-M0 and 34,560 bytes on the Cortex-M4F. The AES-128-GCM marginal Flash cost is comparatively stable across levels, between roughly 16,000 and 26,000 bytes. The shape of the Ascon-AEAD128 curve is the same on both boards, rising from `-O0` through a peak at `-O3` and then falling sharply at `-Os`.

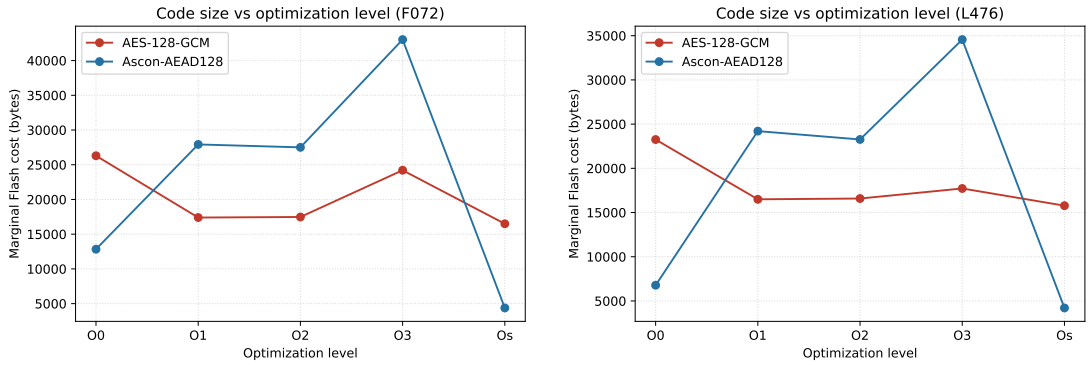

Figure 4.5: Marginal Flash cost across all optimization levels. Left: Cortex-M0 (F072). Right: Cortex-M4F (L476). The Ascon-AEAD128 cost varies strongly with optimization level due to permutation unrolling, while AES-128-GCM remains comparatively stable.

4.6 Summary of Results

Across both boards and all message sizes, Ascon-AEAD128 executed faster than AES-128-GCM at every optimization level. The advantage was smallest at `-O0` and larger once optimization was enabled. At equal clock frequency the advantage was consistently larger on the Cortex-M0 than on the Cortex-M4F, reaching a factor of about 4.6 at 1000 bytes on the Cortex-M0. In code size, Ascon-AEAD128 was substantially smaller than AES-128-GCM at `-Os`, by a factor of about 3.8 on both boards, while at the speed-oriented optimization levels its reference implementation was larger than AES-128-GCM due to the expansion of the permutation code. These results are interpreted in relation to the four research questions in Chapter 5.

5 Discussion

5.1 Performance (Research Question 1)

The first research question asks how the two algorithms compare in execution time and throughput across message sizes on each platform. The measurements give a clear answer: Ascon-AEAD128 was faster than AES-128-GCM at every measured message size, on both boards, and at every optimization level. The advantage was present both in the fixed per-call cost that dominates small messages and in the per-byte cost that dominates large messages.

The two comparisons are worth separating, because the relevant one depends on the message size a deployment actually handles. For short messages the fixed per-call cost is decisive. At a single byte and `-O2`, Ascon-AEAD128 cost 7,410 cycles against 25,185 cycles for AES-128-GCM on the Cortex-M0, and 4,219 against 6,470 cycles on the Cortex-M4F. For long messages the per-byte rate is decisive. In the large-message region at `-O2`, Ascon-AEAD128 reached approximately 169 cycles per byte against 771 for AES-128-GCM on the Cortex-M0, and approximately 95 against 189 cycles per byte on the Cortex-M4F. Ascon-AEAD128 therefore leads in both comparisons, so the conclusion does not depend on the message-size profile of the application.

The reason for the difference lies in the structure of the two algorithms. AES in software relies on table lookups for the substitution step, and the GCM authentication step performs a multiplication in a binary field, built from many smaller operations when no wide hardware multiplier is available. Ascon-AEAD128 is built from a permutation of bitwise operations — exclusive-or, bitwise-and, and rotation — applied to a small state held in registers. These operations are native to both cores and require no table memory. The single-pass duplex construction also avoids the separate encryption and authentication passes that AES-128-GCM performs. The measured advantage is the practical consequence of this design difference.

5.2 Memory (Research Question 2)

The second research question asks about the Flash footprint of each implementation relative to the constraints of a small microcontroller. The answer depends on the optimization level, and the size-optimized level is the one relevant to a memory-constrained deployment.

At `-Os`, Ascon-AEAD128 added 4,376 bytes of Flash on the Cortex-M0 and 4,208 bytes on the Cortex-M4F, while AES-128-GCM added 16,500 and 15,776 bytes respectively. AES-128-GCM therefore required approximately 3.8 times the code space of Ascon-AEAD128, consistently on both boards. On the Cortex-M0, which has 128 KB of Flash, the difference between roughly 4 KB and roughly 16 KB is a meaningful fraction of the budget once application code is also present. The small code size of Ascon-AEAD128 follows from its single permutation and its lack of precomputed lookup tables, whereas AES-128-GCM brings the AES tables and the

field-multiplication code.

It must be noted that this advantage at `-Os` does not hold at the speed-oriented optimization levels, for reasons examined in Section 5.3. The footprint result is therefore reported at the optimization level that a constrained deployment would actually choose, where Ascon-AEAD128 is both smaller and, from the timing results, faster.

5.3 Optimization Sensitivity (Research Question 3)

The third research question asks how the compiler optimization level affects the relative performance of the two algorithms. The measurements show that the optimization level matters a great deal, in two distinct ways.

The first concerns execution time. The largest single improvement for both algorithms came from enabling optimization at all — the step from `-O0` to `-O1`. At 1000 bytes on the Cortex-M0 this step reduced the Ascon-AEAD128 time by a factor of about eight and the AES-128-GCM time by a factor of about three. This uneven response means the relative advantage of Ascon-AEAD128 is understated by any measurement taken at `-O0`. A study that benchmarked only unoptimized code would have reported the two algorithms as close — with a ratio of 1.16 on the Cortex-M4F — and would have missed the larger advantage that appears under the optimization a real build would use. The choice to sweep the optimization level, rather than report a single level, is justified by this observation.

The behavior at the higher levels was not uniform. On the Cortex-M4F, AES-128-GCM gained nothing from `-O3` over `-O2`, consistent with the common observation that aggressive optimization yields little for table-driven code on these cores, while Ascon-AEAD128 continued to improve at `-O3`. On the Cortex-M0 the pattern was reversed, with AES-128-GCM improving substantially at `-O3` and Ascon-AEAD128 improving only modestly. The `-Os` level was slower than `-O2` for both algorithms on both boards, because size optimization disables the inlining and loop unrolling that the cryptographic inner loops benefit from.

The second way the optimization level matters concerns code size, and it is the more striking effect. The Flash cost of the Ascon-AEAD128 reference implementation varied strongly with the optimization level, while the AES-128-GCM cost was comparatively stable. The Ascon-AEAD128 cost rose from `-O0` to a peak at `-O3`, reaching 43,024 bytes on the Cortex-M0 and 34,560 bytes on the Cortex-M4F, then fell sharply at `-Os` to roughly 4 KB. The cause is the unrolling of the permutation: at the speed-oriented levels the compiler unrolls the permutation rounds into straight-line code, which the authenticated-encryption routine includes at several call sites, expanding the binary. At `-Os` the compiler keeps the rounds rolled, and the code collapses to a small size. The same shape appeared on both boards, showing that the effect is a property of the implementation and the compiler rather than of the processor.

This links the second and third research questions. The footprint advantage of Ascon-AEAD128 is real but conditional on the optimization level: it exists at `-Os` and is reversed at the speed-oriented levels. The advantage and this condition should be reported together.

5.4 Architecture Dependence (Research Question 4)

The fourth research question asks whether the performance gap between the two algorithms differs between the Cortex-M0 and the Cortex-M4F, and what this implies for device selection. The measurements show that the gap is consistently larger on the weaker core.

At equal clock frequency and at 1000 bytes, the ratio of AES-128-GCM time to Ascon-AEAD128 time was about 4.6 at `-O1` and `-O2` on the Cortex-M0, against about 2.0 at the same levels on the Cortex-M4F. At every optimized level the ratio was larger on the Cortex-M0. The advantage of Ascon-AEAD128 therefore grows as the processor becomes more constrained.

This can be read against the architectural baseline established in the harness validation. For the plain memory workload the Cortex-M0 was about 2.8 times slower than the Cortex-M4F at the same clock. If both algorithms simply scaled with this architectural factor, their ratio to each other would be the same on both boards. The fact that the AES-128-GCM disadvantage grows by more than this factor on the Cortex-M0 indicates that AES-128-GCM contains operations that are especially costly on the weaker core. The field multiplication in the GCM authentication step is the principal example: it must be synthesized from many small operations when no wide hardware multiplier is present, which is the case on the Cortex-M0. Ascon-AEAD128, built from operations the Cortex-M0 executes natively, stays closer to the architectural baseline.

The implication for device selection is direct: the more constrained the processor, the stronger the case for Ascon-AEAD128 over AES-128-GCM. On the lowest-end cores, which are also the most numerous and the most cost-sensitive, the advantage is largest.

5.5 The Fairness of the Comparison

The comparison was designed to avoid handicapping either algorithm, and one configuration choice deserves explicit defense. AES-128-GCM was measured with the small field-multiplication table, as disclosed in Section 3.4.1. A larger table exists that would improve the AES-128-GCM throughput, at a cost of roughly 3.8 KB of additional RAM that Ascon-AEAD128 does not require. The small table was chosen because it represents a genuinely lightweight configuration of AES-128-GCM, which matches the constrained-device setting of this work. On a part with 16 KB of SRAM, spending close to 4 KB of RAM on an acceleration table is exactly the kind of cost a constrained deployment may decline.

The reported AES-128-GCM figures therefore represent the memory-frugal end of AES-128-GCM rather than its fastest possible configuration. Enabling the larger table would narrow the throughput gap at large messages, at a RAM cost Ascon-AEAD128 does not pay, and would not change the code-size or small-message conclusions. The comparison is thus a fair one between two lightweight configurations, and the configuration choice is named and justified so the result cannot be read as a weakened AES-128-GCM.

5.6 Practical Conclusions

The results support a consistent practical recommendation for the class of devices studied. For authenticated encryption on a constrained microcontroller without hardware AES acceleration,

Ascon-AEAD128 is the stronger choice on the measured criteria: it was faster than AES-128-GCM at every message size and optimization level, smaller in code at the size-optimized level that such a device would use, and its advantage was largest on the most constrained processor.

Two qualifications accompany this recommendation. First, the speed advantage and the size advantage do not occur at the same optimization level for the reference implementation. The size advantage holds at `-Os`, where Ascon-AEAD128 is both smaller and faster than AES-128-GCM, so a deployment that builds for size obtains both benefits at once. A deployment that builds for speed at `-O2` or `-O3` obtains the speed advantage but accepts a larger Ascon-AEAD128 code size due to the unrolling of the permutation. Second, these conclusions concern software-only implementations. On a device with a hardware AES accelerator the balance would differ, and that case was deliberately outside the scope of this work in order to isolate the software comparison.

Within those qualifications, the measurements give a quantitative basis for preferring Ascon-AEAD128 on constrained, software-only targets, and they quantify how the advantage grows as the device becomes more limited.

6 Conclusion and Outlook

6.1 Conclusion

This work presented a measurement-based comparison of Ascon-AEAD128 and AES-128-GCM on two constrained ARM Cortex-M microcontrollers — a Cortex-M0 and a Cortex-M4F — using software-only implementations under matched conditions. The study was motivated by the absence of quantitative performance data for the standardized lightweight algorithm on representative constrained hardware, and it set out to provide that data across execution time, code size, optimization level, and processor architecture.

The measurements answer the four research questions consistently. On execution time, Ascon-AEAD128 was faster than AES-128-GCM at every message size and every optimization level on both boards, in both the fixed per-call regime that governs short messages and the per-byte regime that governs long messages. On code size, Ascon-AEAD128 required about one quarter of the Flash of AES-128-GCM at the size-optimized level that a memory-constrained deployment would use. On optimization sensitivity, the relative advantage of Ascon-AEAD128 was understated at no optimization and clearer once optimization was enabled, and the code size of the Ascon-AEAD128 reference implementation was found to vary strongly with the optimization level because of the unrolling of its permutation. On architecture dependence, the advantage of Ascon-AEAD128 was consistently larger on the more constrained Cortex-M0 than on the Cortex-M4F, indicating that the case for Ascon-AEAD128 strengthens as the target device becomes more limited.

Taken together, the results give a quantitative basis for preferring Ascon-AEAD128 over AES-128-GCM for authenticated encryption on constrained, software-only targets, subject to the qualification that the speed advantage and the code-size advantage are obtained together only when building for size. The work thereby contributes the kind of measurement-based evidence that was missing for this comparison on this class of hardware.

6.2 Outlook

Several directions extend this work.

The first concerns the AES-128-GCM configuration. The measurements used the memory-frugal small-table configuration, as disclosed in Section 3.4.1. A measurement of the larger-table configuration would bound the fastest AES-128-GCM throughput achievable on these boards and would quantify the speed gained for the additional RAM, completing the picture of the speed-against-memory trade-off on the AES-128-GCM side.

The second concerns operating frequency. The timing comparison was made at a matched 48 MHz so that the architectural difference could be isolated. Because the results are reported

as cycle counts they are independent of the operating frequency, but a measurement of the Cortex-M4F at its native 80 MHz would confirm this directly and would give wall-clock figures representative of that board running at full speed.

The third concerns the hashing primitives. This work compared the authenticated encryption primitives only. A comparison of Ascon-Hash256 against SHA-256 on the same hardware would extend the lightweight comparison to the hashing use case, which arises in key derivation and integrity checking on the same devices.

The fourth concerns hardware acceleration. The comparison was deliberately restricted to software-only implementations in order to isolate the algorithmic difference. On a device with a hardware AES accelerator the balance would shift toward AES-128-GCM for the encryption step, while the authentication step and the code-size considerations would remain. A study on a part that includes a hardware AES block would characterize this case and would mark the boundary of the recommendation made here.

The fifth concerns post-quantum readiness. The ASCON family includes a variant with a longer key, Ascon-80pq, intended to raise the security margin against adversaries with greater computational capability [7]. Because it shares the same permutation as Ascon-AEAD128, its performance on constrained hardware is expected to be close, and a measurement would confirm the cost of the larger key. This would position the lightweight comparison with respect to longer-term security requirements.

Each of these directions builds on the measurement infrastructure established here, which was validated, frozen, and documented for reproducibility, so that further measurements can be added under identical conditions and compared directly with the results reported in this work.

Bibliography

[1] National Institute of Standards and Technology, "Advanced Encryption Standard (AES)," National Institute of Standards and Technology, Federal Information Processing Standard 197, 2001.

[2] ——, "Submission Requirements and Evaluation Criteria for the Lightweight Cryptography Standardization Process," National Institute of Standards and Technology, Tech. Rep., 2018, available at https://csrc.nist.gov/projects/lightweight-cryptography.

[3] ——, "NIST Selects 'Ascon' for Lightweight Cryptography Standardization," NIST Announcement, February 2023, 2023, https://www.nist.gov/news-events/news/2023/02/nist-selects-ascon-lightweight-cryptography-standardization.

[4] M. Sönmez Turan, K. A. McKay, D. Chang, J. Kang, and J. Kelsey, "Ascon-Based Lightweight Cryptography Standards for Constrained Devices: Authenticated Encryption, Hash, and Extendable Output Functions," National Institute of Standards and Technology, NIST Special Publication 800-232, Aug. 2025.

[5] M. J. Dworkin, "Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC," National Institute of Standards and Technology, NIST Special Publication 800-38D, 2007.

[6] C. Dobraunig, M. Eichlseder, F. Mendel, and M. Schläffer, "ASCON reference software (ascon-c)," GitHub, 2023, https://github.com/ascon/ascon-c, commit `446347f21b209f3921c65ece70027c366cbe1693`.

[7] ——, "ASCON v1.2," Submission to the NIST Lightweight Cryptography Standardization Project, 2021, https://ascon.iaik.tugraz.at/.

[8] Mbed TLS Contributors, "Mbed TLS," GitHub, 2024, https://github.com/Mbed-TLS/mbedtls, commit `107ea89daaefb9867ea9121002fbbdf926780e98`.